

Homework #2

Algorithm

I followed the algorithm given in class.

Algorithm 1 Deep Q-Network

```
1: Initialize Q-network weights  $\theta$ 
2: Initialize the replay buffer  $\mathcal{B}$  with capacity  $D$ .
3: Initialize target network  $\hat{Q}_{\theta'}$  weights  $\theta' = \theta$ 
4: for episode=1, ...,  $M$  do
5:   for  $t = 1, \dots, T$  do
6:     Based on the current state  $s_t$ , choose action  $a_t$  according to  $\epsilon$ -Greedy policy.
7:     Take action  $a_t$ , collect a tuple  $(s_t, a_t, r_t, s_{t+1})$  and add to the replay buffer  $\mathcal{B}$ .
8:     Randomly sample a mini-batch of tuples  $\mathcal{S} = \{(s_i, a_i, r_i, s_{i+1})\}$  from the replay buffer  $\mathcal{B}$ .
9:     for each tuple  $(s_i, a_i, r_i, s_{i+1})$  in the batch  $\mathcal{S}$  do
10:      Compute  $y_i$ :
11:      if episode terminates after step  $i$  then
12:         $y_i = r_i$ 
13:      else
14:         $y_i = r_i + \gamma \max_{a'} \hat{Q}_{\theta'}(s_{i+1}, a')$  using the target network with weights  $\theta'$ 
15:      end if
16:    end for
17:    Update  $\theta$  by taking an optimization step on the loss  $L(\theta) = \sum_i (Q_{\theta}(s_i, a_i) - y_i)^2$ 
18:    Update  $\theta'$  after every  $N$  optimization steps.
19:  end for
20: end for
```

I also implemented discounted update of the target network's weights as such:

$$\phi': (1 - \tau)\phi' + \tau\phi \rightarrow \phi'$$

Where

τ is a very small number, and ϕ correspond to the weights of the policy network, and ϕ'

Correspond to the weights of the target network.

For the implementation, I tested both tensorflow and PyTorch. However, for the final implementation, I chose PyTorch. For the implementation, I referred to PyTorch tutorials regarding data structures and types and functions that are recommended in Pytorch. I also referred to a youtube tutorial regarding implemnetaing the epsilon greedy algorithm. I tested several hyperparameters. I also set an initiaization method for the weights of the neural networks (He).

Also, in my code, I update the target neural network after every optimization step of the policy network (Q). I tested updating the target network weights after 100, 50, and 10 episodes, and none led to learning at an appropriate pace. Moreover, since I used discounted learning (Using τ), that matches the weights of tagret network to that of the policy

network(Q), this ensures that the learning is stable; so the target network's weights don't match the weights of the Q network exactly.

Final Run Results

After finishing the training using file (HW2_p1_Torch_run4.ipynb), we obtain the following files:

- np_eps_rewards_run4.npy: contains the list of episode rewards
- policy_net_cartpole_state_dict_v4.pth: contains the state dictionary of the policy network model.
- policy_net_cartpole_v4.pth: contains the full architecture and weights of the policy network model. We use this trained model to roll out 500 episodes and obtain a histogram.
- Qmax_episodes_nps_run4.npy: the list of average Qmax values for each training episode.
- sample500_np_eps_rewards_cartpole.npy: the list of rewards of a sample of 500 episodes using the trained policy network model.

Figure 1 shows the plot of episode rewards versus the training episodes. We notice that the algorithm almost converges before the episode 200. However, I keep training the model, and the model shows some fluctuation in the training, as expected in neural networks due to catastrophic forgetting, but then it converges again before the 600th training episode. I could have stopped training at episode 250 alternatively when the last 100 episodes obtained an average >195. This graph shows that the model is learning fast and adequately.

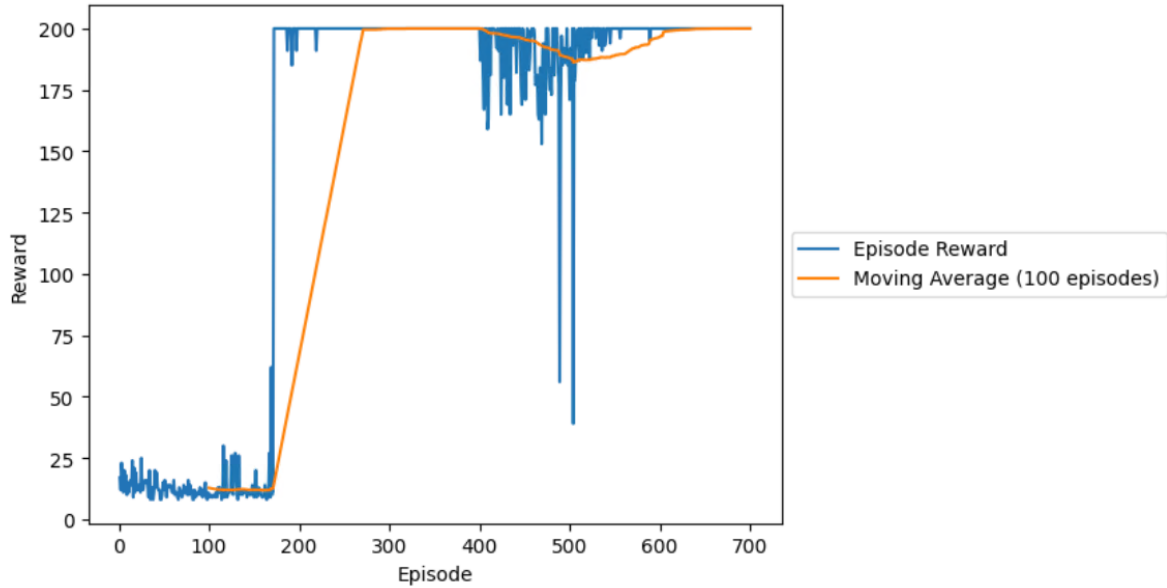


Figure 1: Episode Rewards plot for final run

Figure 2 shows the average Qmax value for each episode of the training episodes. We also find that Q value is increasing, and it converges close to episode 200.

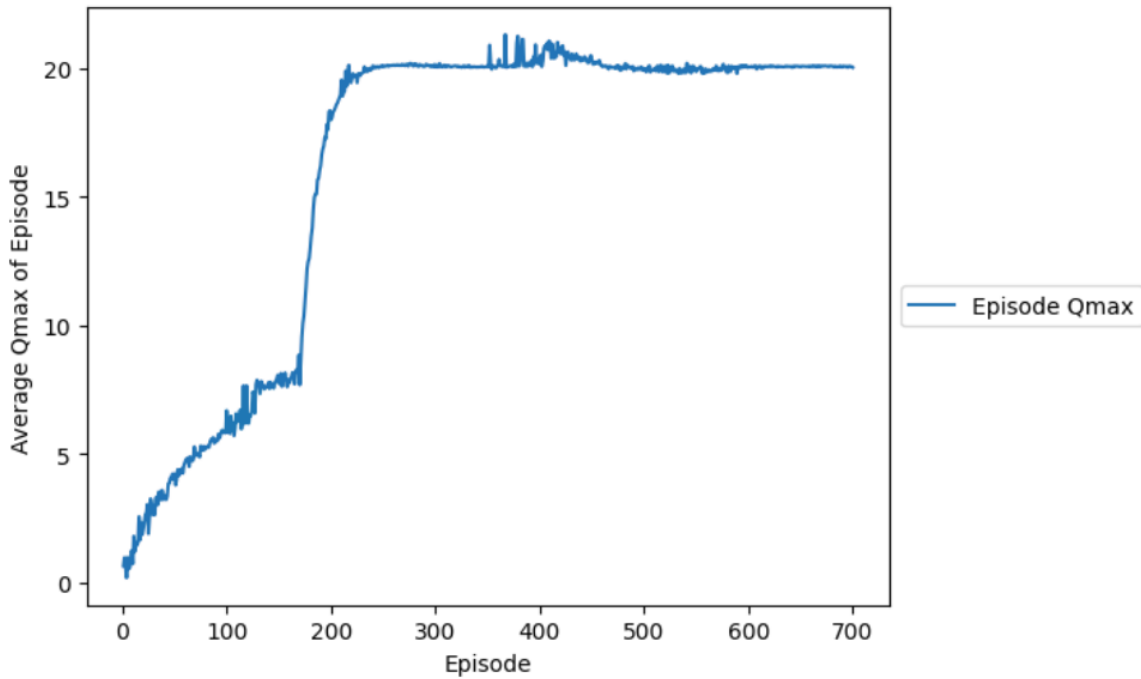


Figure 2: Average Qmax plot during training episodes for final run

Next, we roll out 500 episodes and choose the action using the trained policy network that we saved after we finished training. We use the code: `hw2-p1-plottingsample.ipynb`. And we obtain the histograms in Figure 3 that shows 500 episodes (but ymax is limited to 1, so t

shows 500 bars), and in Figure 4, where ymax is 500, and we find one bar only since all the 500 episodes obtained a reward of 500. And the mean is 200 and standard deviation is 0.

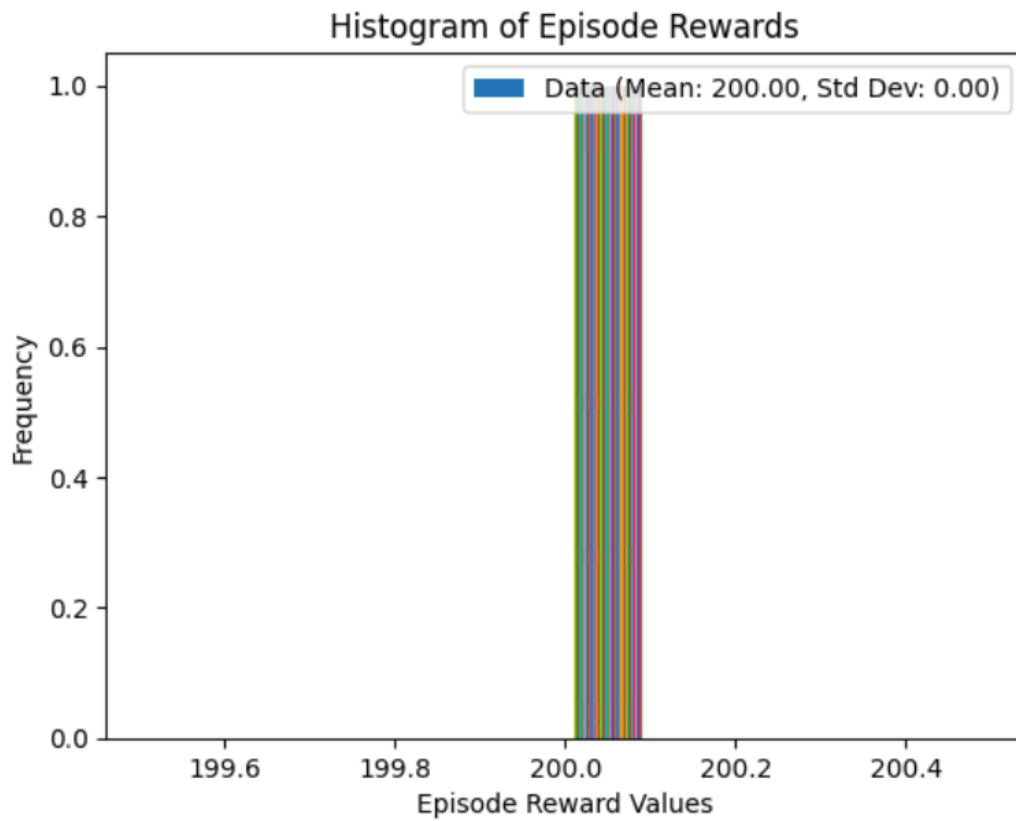


Figure 3: Histogram of Episode Rewards of a sample of 500 episodes using trained model (frequency limit of 1)

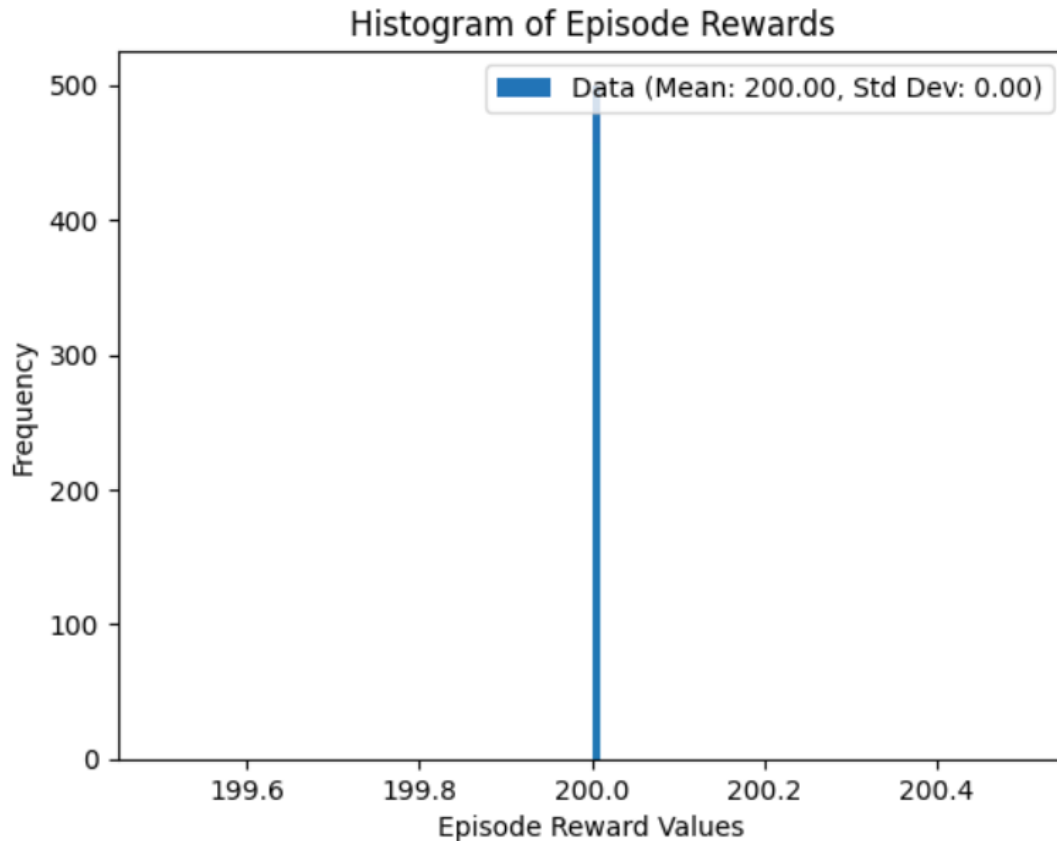


Figure 4: Histogram of Episode Rewards of a sample of 500 episodes using trained model (frequency limit of 500)

Previous Runs

While testing the model and running several runs initially, we obtained different outcomes that looked too good at times, and less good at other times. This shows that the performance of the DQN depends on the initial seed as mentioned during the lecture. (note that I forgot to plot the Qmax values for the previous runs).

Run 1

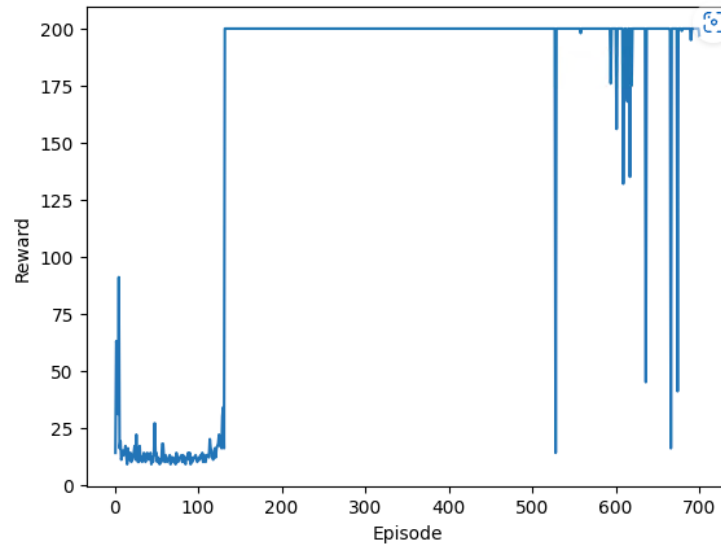


Figure 5: Episode Rewards plot for run 1

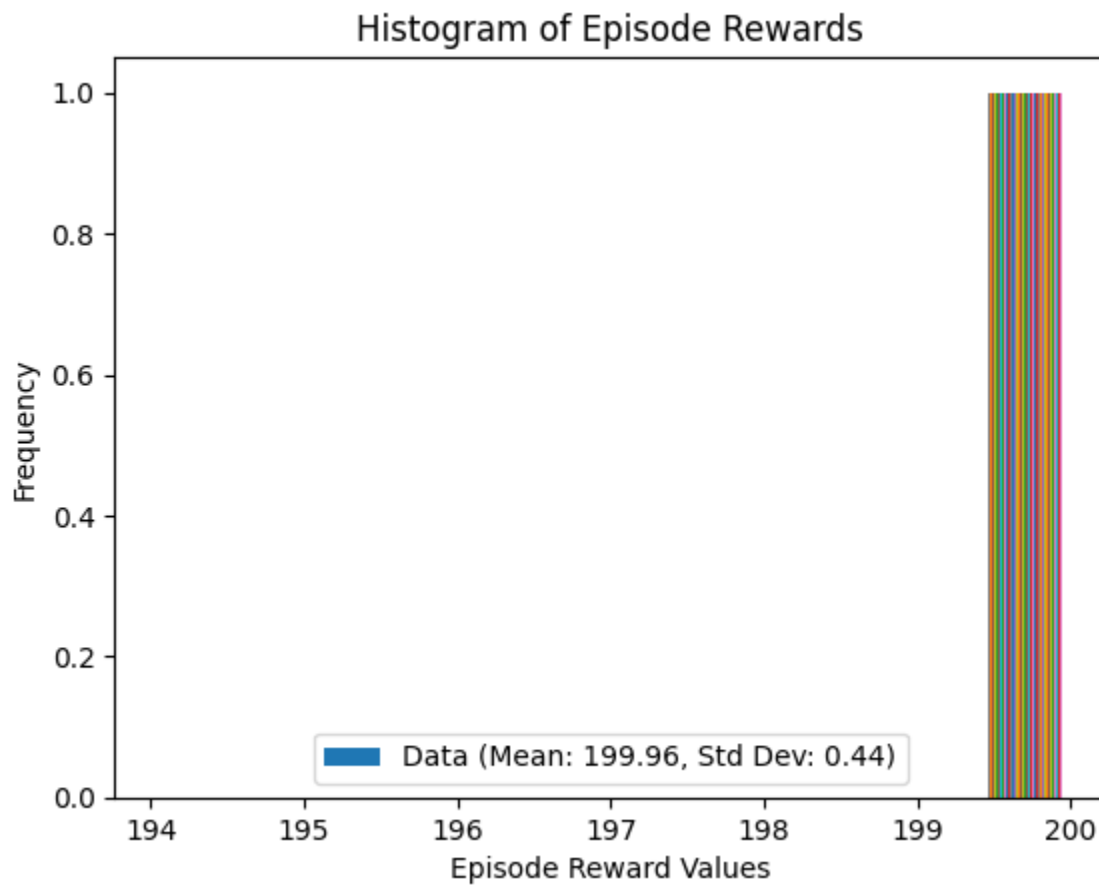


Figure 6: Histogram of Episode Rewards of a sample of 500 episodes using trained model -run1

Run 2

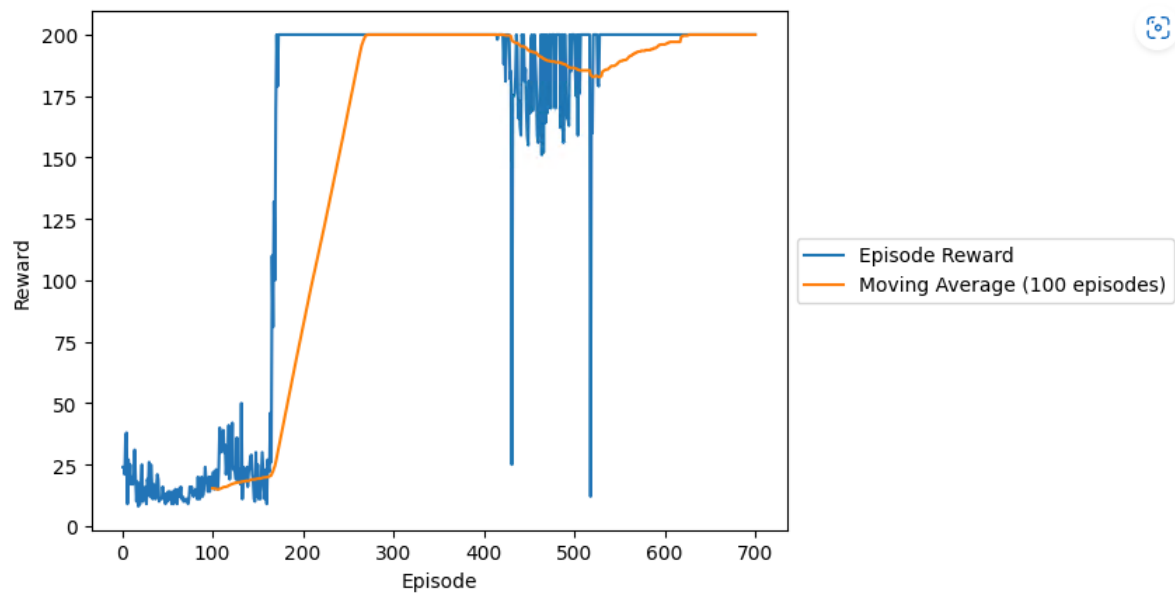


Figure 7: Episode Rewards plot for final run2

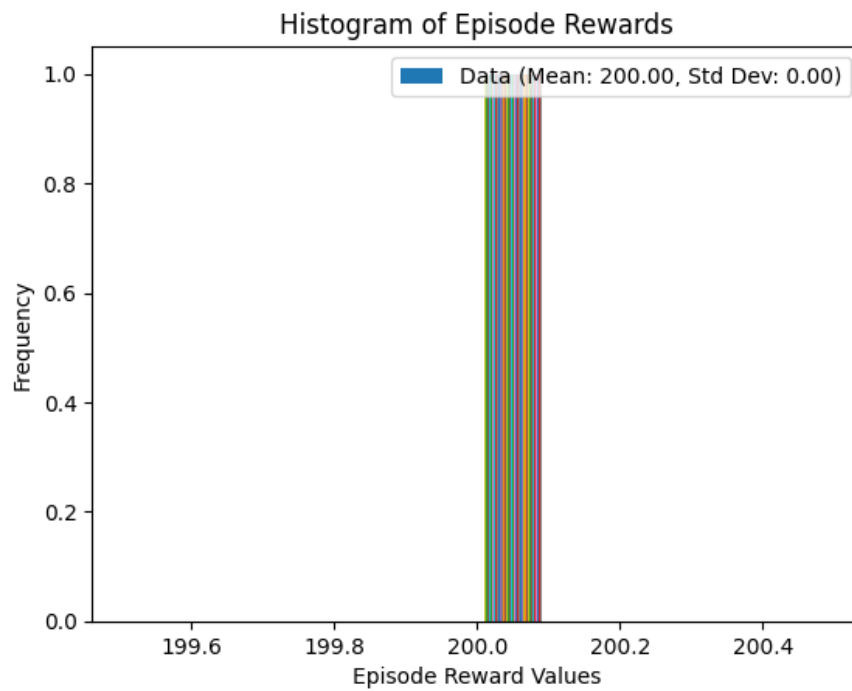


Figure 8: Histogram of Episode Rewards of a sample of 500 episodes using trained model of run2

Run 3

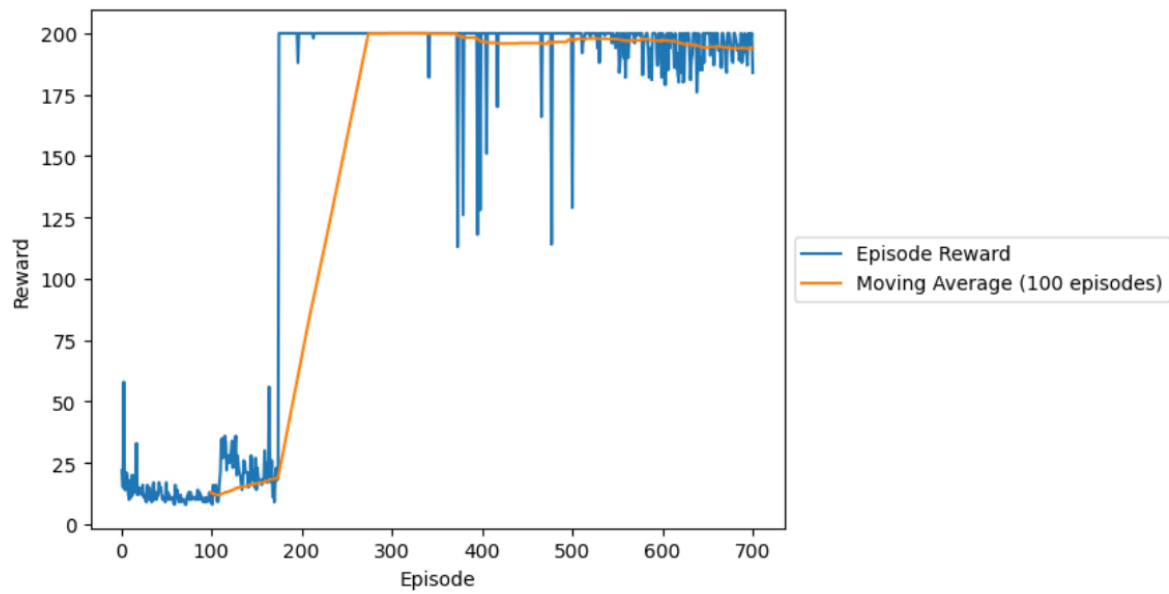


Figure 9: Episode Rewards plot for final run3

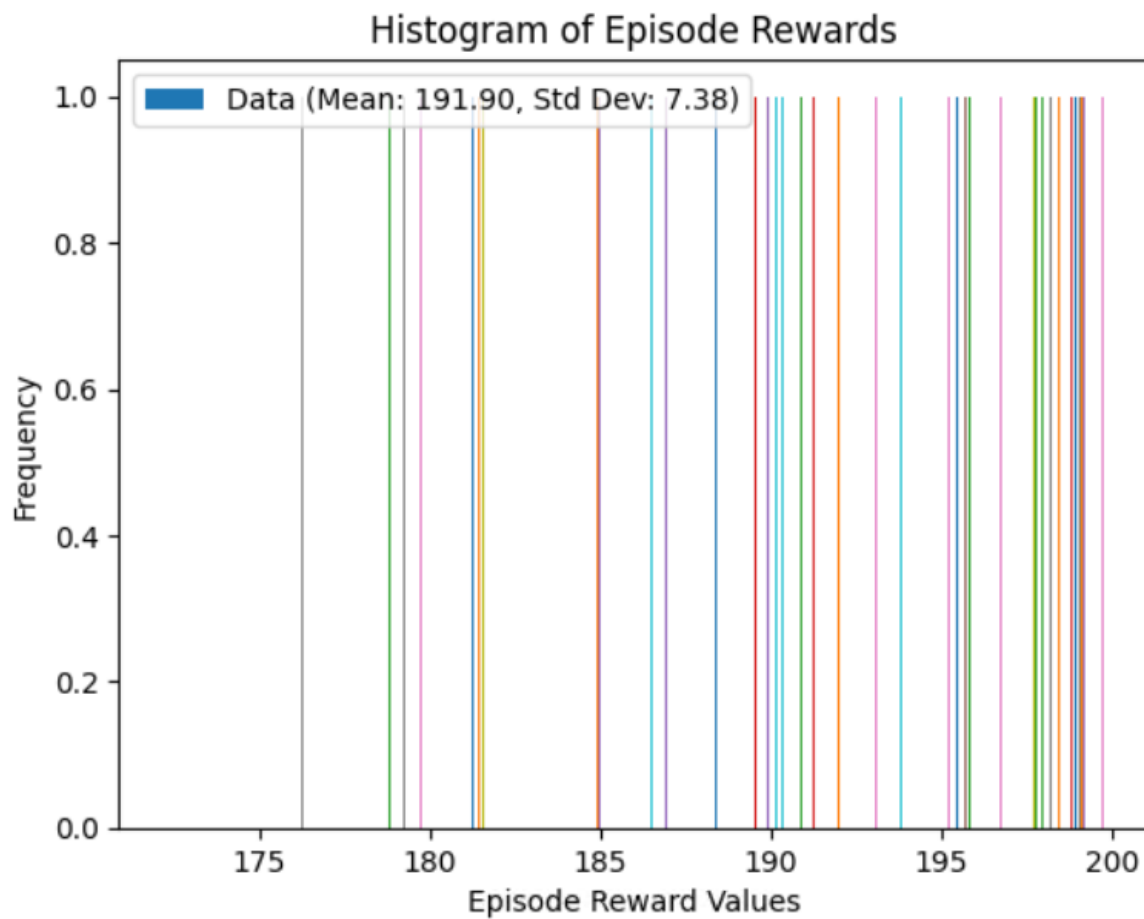


Figure 10: Histogram of Episode Rewards of a sample of 500 episodes using trained model of run3

References

- <https://www.youtube.com/watch?v=EjYESbg95x0>
- PyTorch Tutorial for DQN
- <https://www.nature.com/articles/nature14236>